



Memoria Técnica

Módulo:

Sistemas Informáticos | Bases de Datos | Lenguajes de Marcas | Entornos de Desarrollo | Programación

Ciclo formativo

1.º Desarrollo de Aplicaciones Web

Alumno/a

Carlos Rubiño Selva

Curso

2025/2026

Fecha de entrega

31 de mayo de 2026

Centro de formación

XTART

ÍNDICE

1. Introducción	4
2. Arquitectura General del Sistema.....	5
2.1 Capa de datos — Base de datos	5
2.2 Capa lógica — Backend en Java con Spring Boot.....	5
2.3 Capa de presentación — Frontend web	5
3. Bases de Datos.....	7
3.1 Diseño del modelo de datos	7
3.2 Claves primarias, foráneas y restricciones.....	8
3.3 Diagrama Entidad-Relación (EER)	9
3.4 Gestión de usuarios y permisos de la base de datos.....	10
3.5 Procedimientos almacenados.....	11
Procedimiento 1: ContarMatches	11
Procedimiento 2: CargarUsuario	12
Procedimiento 3: BorrarUsuario	12
3.6 Trigger automático de match	12
3.7 Inserción de datos de prueba	14
3.8 Consultas SQL desarrolladas	15
Consulta 1 — JOIN: usuarios con sus tecnologías.....	15
Consulta 2 — COUNT + GROUP BY: número de tecnologías por usuario	16
Consulta 3 — ORDER BY: usuarios por fecha de registro	17
Consulta 4 — LEFT JOIN: usuarios sin tecnologías asignadas.....	18
Consulta 5 — Matches con nombres de ambos usuarios	19
Consulta 6 — Ranking de tecnologías más populares	20
Consulta 7 — Likes enviados por usuario	21
Consulta 8 — Vista vista_perfiles.....	22
4. Programación (Java con Spring Boot).....	24
4.1 Estructura del proyecto.....	24
4.2 Conexión con la base de datos.....	25
4.3 Endpoints de la API REST	26
4.4 Pantalla de inicio de sesión	26
4.5 Registro de nuevos usuarios	27
4.6 Vista de perfiles y sistema de likes	28
4.7 Detección del match y ficha de compatibilidad	29
5. Lenguajes de Marcas (HTML, CSS y JavaScript)	31
5.1 Identidad visual y diseño de la interfaz	31
5.2 Página de inicio de sesión (index.html).....	31
5.3 Página de registro (registro.html).....	31

5.4	Página de perfiles (perfiles.html)	31
5.5	Comunicación asíncrona con el backend	32
6.	Entornos de Desarrollo (Git y GitHub)	33
6.1	Estructura del repositorio	33
6.2	Problema con GitHub Desktop y solución adoptada	34
7.	Sistemas Informáticos	35
7.1	De la máquina virtual a Docker	35
7.2	Sistema de copias de seguridad	36
	Backup manual con DBeaver	36
	Script de backup automático (.bat)	37
7.3	Backup avanzado con volúmenes Docker	37
7.4	Automatización mediante tareas programadas	38
8.	Problemas Encontrados y Soluciones Adoptadas	39
8.1	Pérdida de la máquina virtual Ubuntu	39
8.2	Errores de dependencias en el archivo pom.xml	39
8.3	Errores en las consultas SQL por nombres de columna incorrectos	40
8.4	Repositorio no detectado por GitHub Desktop	40
9.	Conclusiones	41
10.	Referencias	43

1. Introducción

LoveCode es una aplicación web desarrollada durante la tercera evaluación del ciclo formativo de Desarrollo de Aplicaciones Web (DAW). La idea principal es simular una plataforma de emparejamiento orientada al mundo del desarrollo de software: en vez de conectar personas por compatibilidad romántica, conecta programadores y desarrolladores que comparten tecnologías e intereses técnicos similares.

El funcionamiento básico de la aplicación es el siguiente. Cualquier persona puede registrarse indicando su nombre, una breve descripción de su perfil y las tecnologías que conoce o le interesan. Una vez registrado, el usuario puede explorar los perfiles de otros desarrolladores y darles "like" si le resultan interesantes. Cuando dos usuarios se dan like mutuamente, el sistema genera automáticamente un "match" entre ellos y muestra una ficha de compatibilidad con sus datos personales y las tecnologías que tienen en común.

Lo que hace especial a este proyecto es que no se trata simplemente de una aplicación aislada, sino que integra de forma coordinada cinco asignaturas del ciclo formativo, cada una aportando una parte esencial del sistema:

- Bases de Datos: diseño y gestión de la base de datos relacional con MariaDB/MySQL, donde se almacena toda la información del sistema.
- Programación: desarrollo del servidor backend en Java con Spring Boot, que gestiona la lógica del negocio y comunica el frontend con la base de datos.
- Lenguajes de Marcas: creación de la interfaz web con HTML5, CSS3 y JavaScript, que es lo que el usuario ve e interactúa en el navegador.
- Entornos de Desarrollo: gestión del código fuente mediante Git y GitHub, herramientas fundamentales en cualquier proyecto de desarrollo profesional.
- Sistemas Informáticos: gestión del entorno de ejecución mediante Docker y sistema de copias de seguridad automatizadas de la base de datos.

El desarrollo se ha organizado en dos fases. La primera fase se centró en construir los cimientos del sistema: diseñar la base de datos, establecer la conexión desde Java y crear las primeras páginas visuales. La segunda fase se dedicó a completar toda la funcionalidad: el sistema de likes y matches, la generación de fichas de compatibilidad y la automatización de las tareas de mantenimiento del sistema.

A lo largo del desarrollo surgieron varios problemas técnicos que fue necesario resolver, como la pérdida de la máquina virtual donde se alojaba la base de datos, errores de compatibilidad entre librerías de Java o fallos en las consultas SQL. Todos estos inconvenientes se superaron y forman parte del aprendizaje real que ofrece este tipo de proyectos.

2. Arquitectura General del Sistema

Para entender cómo está construido LoveCode es útil conocer primero su arquitectura, es decir, cómo están organizadas y conectadas las distintas partes del sistema. LoveCode sigue una arquitectura de tres capas, que es el modelo más habitual en las aplicaciones web modernas. Cada capa tiene una responsabilidad bien definida y se comunica con las demás de una manera ordenada.

2.1 Capa de datos — Base de datos

La capa de datos es el almacén de toda la información de la aplicación. Aquí se guardan los perfiles de los usuarios, las tecnologías disponibles, los likes que se dan entre ellos y los matches que se generan. Esta información se organiza en tablas relacionadas entre sí mediante claves primarias y foráneas, formando lo que se conoce como una base de datos relacional.

Para LoveCode se utiliza MySQL 8.0 como sistema gestor de base de datos, ejecutándose dentro de un contenedor Docker. La decisión de usar Docker en lugar de una instalación convencional ofrece importantes ventajas que se explican en detalle en el apartado de Sistemas Informáticos.

2.2 Capa lógica — Backend en Java con Spring Boot

La capa lógica, también llamada backend, es el "cerebro" de la aplicación. Es la parte que no ve el usuario directamente, pero que hace que todo funcione correctamente. El servidor está desarrollado en Java utilizando el framework Spring Boot, que simplifica enormemente la creación de servidores web en Java al proporcionar configuraciones predeterminadas y una estructura de proyecto bien definida.

El backend recibe las peticiones que envía el navegador del usuario, decide qué hacer con ellas (consultar la base de datos, validar credenciales, registrar un like, etc.), ejecuta la operación correspondiente y devuelve la respuesta al navegador en formato JSON, que es un formato de texto estructurado ampliamente utilizado en aplicaciones web para intercambiar datos.

2.3 Capa de presentación — Frontend web

La capa de presentación, también llamada frontend, es todo lo que el usuario ve e interactúa en su navegador: los formularios, las tarjetas de perfil, los botones y los mensajes de la aplicación. Está formada por tres tecnologías complementarias: HTML5 para definir la estructura y el contenido de las páginas, CSS3 para aplicar el diseño visual y los estilos, y JavaScript para añadir interactividad y comunicarse con el servidor.

Gracias a JavaScript, las páginas de LoveCode no se recargan completamente cada vez que el usuario realiza una acción. En cambio, JavaScript envía peticiones HTTP asíncronas al servidor en

segundo plano y actualiza solo la parte de la página que necesita cambiar, ofreciendo así una experiencia de usuario mucho más fluida y moderna.

El flujo completo de la información es el siguiente: el usuario interactúa con la interfaz web (frontend), que envía una petición al servidor Java (backend), que consulta o modifica la base de datos según sea necesario y devuelve el resultado al frontend, que lo muestra al usuario de forma visual. Este ciclo se repite con cada acción que realiza el usuario en la aplicación.

3. Bases de Datos

El módulo de Bases de Datos es el pilar fundamental sobre el que se construye todo el proyecto. Sin una base de datos bien diseñada y correctamente implementada, el resto del sistema no podría funcionar. En este apartado se detalla todo el trabajo realizado en este módulo: el diseño inicial del modelo de datos, la creación de las tablas, la gestión de usuarios y permisos, los procedimientos almacenados, el trigger automático de match y las consultas SQL desarrolladas.

3.1 Diseño del modelo de datos

Antes de crear ninguna tabla ni escribir ningún script SQL, el primer paso es pensar cómo se va a organizar la información. Este proceso se llama modelado de datos y consiste en identificar qué entidades (objetos o conceptos) necesita gestionar el sistema y cómo se relacionan entre sí.

Para LoveCode se identificaron cinco entidades principales, cada una representada por una tabla en la base de datos:

- **Usuarios:** representa a cada persona registrada en la aplicación. Almacena su nombre, dirección de correo electrónico, contraseña, una descripción de su perfil y la fecha en que se registró.
- **Tecnologías:** representa el catálogo de tecnologías disponibles en el sistema, como Java, Python, JavaScript, SQL o React. Cada tecnología tiene un identificador único y su nombre.
- **Usuarios_Tecnologías:** es una tabla intermedia que implementa la relación entre usuarios y tecnologías. Un usuario puede conocer varias tecnologías y una tecnología puede ser conocida por muchos usuarios, lo que en diseño de bases de datos se llama relación de muchos a muchos.
- **Likes:** registra cada like que un usuario da a otro. Almacena quién envió el like (`id_emisor`), quién lo recibió (`id_receptor`) y cuándo se produjo.
- **Matches:** registra las coincidencias mutuas entre usuarios. Cuando dos usuarios se han dado like mutuamente, se crea un registro en esta tabla con los identificadores de ambos y la fecha del match.

A continuación se muestra el script SQL de creación de las tablas con todos sus campos, tipos de datos, claves primarias y claves foráneas:

```

CREATE TABLE Usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    descripcion TEXT,
    fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP
);

>Run | Select
CREATE TABLE Tecnologias (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);

>Run | Select
CREATE TABLE Usuarios_Tecnologias (
    id_usuario INT NOT NULL,
    id_tecnologia INT NOT NULL,
    PRIMARY KEY (id_usuario, id_tecnologia),
    FOREIGN KEY (id_usuario) REFERENCES Usuarios(id) ON DELETE CASCADE,
    FOREIGN KEY (id_tecnologia) REFERENCES Tecnologias(id) ON DELETE CASCADE
);

CREATE TABLE Likes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_emisor INT NOT NULL,
    id_receptor INT NOT NULL,
    fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY unico_like (id_emisor, id_receptor),
    FOREIGN KEY (id_emisor) REFERENCES Usuarios(id) ON DELETE CASCADE,
    FOREIGN KEY (id_receptor) REFERENCES Usuarios(id) ON DELETE CASCADE
);

>Run | Select
CREATE TABLE Matches (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario1 INT NOT NULL,
    id_usuario2 INT NOT NULL,
    fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY unico_match (id_usuario1, id_usuario2),
    FOREIGN KEY (id_usuario1) REFERENCES Usuarios(id) ON DELETE CASCADE,
    FOREIGN KEY (id_usuario2) REFERENCES Usuarios(id) ON DELETE CASCADE
);

```

3.2 Claves primarias, foráneas y restricciones

El diseño de las relaciones entre tablas es uno de los aspectos más importantes de cualquier base de datos relacional. Las claves primarias y foráneas son los mecanismos que permiten vincular la información de distintas tablas de forma coherente.

Una clave primaria es el identificador único de cada fila de una tabla, similar al DNI de una persona: no puede haber dos filas con el mismo valor y siempre debe tener un valor. En LoveCode, todas las tablas principales utilizan un campo numérico autoincremental como clave primaria.

Una clave foránea es una columna que hace referencia a la clave primaria de otra tabla, creando así el vínculo entre ellas. Por ejemplo, la tabla Likes tiene dos claves foráneas: `id_emisor` e `id_receptor`, ambas apuntan al campo `id` de la tabla Usuarios.

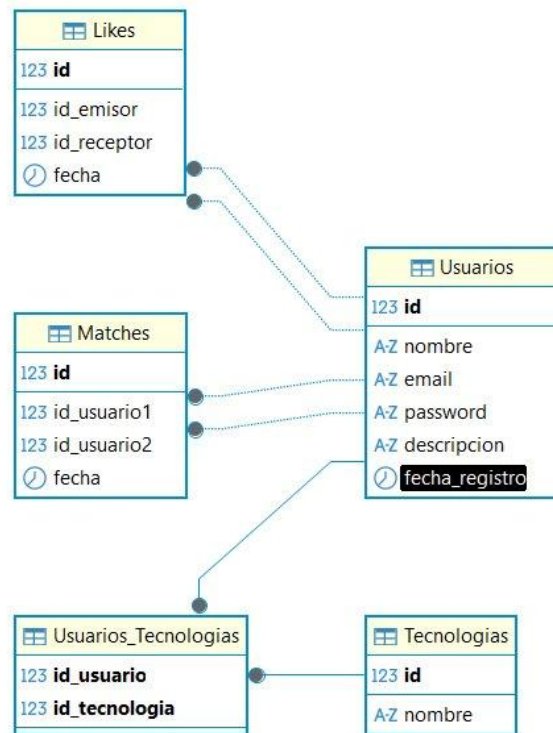
Todas las claves foráneas de LoveCode están configuradas con la opción `ON DELETE CASCADE`. Esto significa que si se elimina un usuario de la base de datos, el sistema borrará automáticamente todos sus registros relacionados: sus likes, sus matches y sus tecnologías asignadas. Esta configuración garantiza que nunca queden datos huérfanos que hagan referencia a usuarios inexistentes.

Además de las claves foráneas, se han añadido restricciones `UNIQUE` en puntos clave: en la combinación (`id_emisor`, `id_receptor`) de la tabla Likes para evitar que un usuario pueda dar like dos veces al mismo perfil, y en la combinación (`id_usuario1`, `id_usuario2`) de la tabla Matches para evitar matches duplicados.

3.3 Diagrama Entidad-Relación (EER)

El diagrama entidad-relación es una representación gráfica de la estructura de la base de datos. Muestra todas las tablas del sistema, los campos que contiene cada una y las relaciones que existen entre ellas mediante líneas y símbolos estandarizados. Es el equivalente a los planos de un arquitecto antes de construir un edificio: permite visualizar el diseño completo antes de implementarlo y detectar posibles errores o incoherencias.

El diagrama se ha generado automáticamente con DBBeaver a partir de la base de datos real del proyecto, lo que garantiza que refleja fielmente la estructura implementada:



En el diagrama se pueden apreciar claramente las cinco tablas y sus relaciones. Las líneas con el símbolo de punto en un extremo indican el lado "muchos" de la relación. Por ejemplo, la línea entre Usuarios y Likes tiene el punto en el lado de Likes, indicando que un usuario puede tener muchos likes. La tabla Usuarios_Tecnologías aparece en el centro conectando Usuarios con Tecnologías, implementando la relación muchos a muchos entre ambas.

3.4 Gestión de usuarios y permisos de la base de datos

En cualquier sistema de base de datos profesional es fundamental aplicar el principio de mínimo privilegio: cada usuario debe tener únicamente los permisos necesarios para realizar su trabajo, y no más. Esto reduce el riesgo de que un error o un acceso no autorizado pueda dañar o comprometer los datos del sistema.

Para LoveCode se han creado tres usuarios de base de datos con diferentes niveles de acceso, simulando los roles que existirían en un entorno real de trabajo:

- carlos (administrador): tiene todos los privilegios sobre la base de datos LoveCode. Puede crear y eliminar tablas, modificar su estructura, gestionar usuarios y realizar cualquier operación. Es el usuario que se utiliza para las tareas de administración del sistema.
- desarrollador: tiene permisos para realizar las cuatro operaciones básicas de datos (SELECT para consultar, INSERT para insertar, UPDATE para modificar y DELETE para eliminar) pero no puede cambiar la estructura de las tablas ni crear nuevos objetos. Es el usuario que utiliza el backend de la aplicación para operar con los datos.

- lector: tiene únicamente permiso de lectura (SELECT). No puede modificar ningún dato. Es útil para consultas de análisis o informes que solo necesitan ver la información sin alterarla.

```
CREATE USER IF NOT EXISTS 'carlos'@'%' IDENTIFIED BY '1234';  
▷Run  
CREATE USER IF NOT EXISTS 'desarrollador'@'%' IDENTIFIED BY '1234';  
▷Run  
CREATE USER IF NOT EXISTS 'lector'@'%' IDENTIFIED BY '1234';
```

```
GRANT ALL PRIVILEGES ON LoveCode.* TO 'carlos'@'%;  
▷Run  
GRANT SELECT, INSERT, UPDATE, DELETE ON LoveCode.* TO 'desarrollador'@'%;  
▷Run  
GRANT SELECT ON LoveCode.* TO 'lector'@'%;  
  
▷Run  
FLUSH PRIVILEGES;
```

3.5 Procedimientos almacenados

Un procedimiento almacenado es un conjunto de instrucciones SQL que se guarda dentro de la propia base de datos con un nombre, de forma que se puede ejecutar cuando sea necesario con una sola llamada. Son similares a las funciones en programación: se definen una vez y se reutilizan tantas veces como sea necesario.

El uso de procedimientos almacenados tiene varias ventajas importantes. En primer lugar, centraliza la lógica de negocio en la base de datos, de forma que cualquier aplicación que se conecte a ella puede beneficiarse de esa lógica sin tener que reescribirla. En segundo lugar, mejora el rendimiento, ya que el código SQL se compila y optimiza una sola vez. En tercer lugar, simplifica el mantenimiento, porque los cambios en la lógica solo hay que hacerlos en un sitio.

Se han implementado tres procedimientos almacenados para las operaciones más habituales del sistema:

Procedimiento 1: ContarMatches

Este procedimiento recibe como parámetro de entrada el identificador de un usuario y devuelve el número total de matches que tiene ese usuario en el sistema. Para calcularlo, cuenta los registros de la tabla Matches en los que el usuario aparece, ya sea como primer participante (id_usuario1) o como segundo (id_usuario2), ya que el orden en que se almacenan los usuarios en el match puede variar.

```

DELIMITER $$

▷Run | ⌕Select
CREATE PROCEDURE ContarMatches(IN p_id INT)
BEGIN
    SELECT COUNT(*) AS total_matches
    FROM Matches
    WHERE id_usuario1 = p_id OR id_usuario2 = p_id;
END$$

```

Procedimiento 2: CargarUsuario

Este procedimiento recibe el identificador de un usuario y devuelve toda su información de perfil de forma completa: nombre, email, descripción, fecha de registro y la lista de tecnologías que conoce. Para obtener las tecnologías, el procedimiento realiza dos JOIN con las tablas Usuarios_Tecnologías y Tecnologías, y utiliza la función GROUP_CONCAT para combinar todos los nombres de las tecnologías en una sola cadena de texto separada por comas, lo que facilita enormemente su uso desde el backend.

```

CREATE PROCEDURE CargarUsuario(IN p_id INT)
BEGIN
    SELECT u.id, u.nombre, u.email, u.descripcion, u.fecha_registro,
           GROUP_CONCAT(t.nombre ORDER BY t.nombre SEPARATOR ', ') AS tecnologias
    FROM Usuarios u
    LEFT JOIN Usuarios_Tecnologias ut ON ut.id_usuario = u.id
    LEFT JOIN Tecnologias t          ON t.id = ut.id_tecnologia
    WHERE u.id = p_id
    GROUP BY u.id;
END$$

```

Procedimiento 3: BorrarUsuario

Este procedimiento recibe el identificador de un usuario y lo elimina del sistema. Gracias a la configuración ON DELETE CASCADE de las claves foráneas, al ejecutar este procedimiento no solo se elimina el registro del usuario de la tabla Usuarios, sino que automáticamente se eliminan también todos sus likes, todos sus matches y todas sus tecnologías asignadas en Usuarios_Tecnologías, dejando la base de datos en un estado completamente limpio y consistente.

```

CREATE PROCEDURE BorrarUsuario(IN p_id INT)
BEGIN
    DELETE FROM Usuarios WHERE id = p_id;
END$$

▷Run
DELIMITER ;

```

3.6 Trigger automático de match

Un trigger, o disparador en español, es un mecanismo de la base de datos que ejecuta automáticamente un bloque de código cuando se produce un determinado evento, como insertar, actualizar o eliminar un registro. A diferencia de un procedimiento almacenado, que hay que llamar explícitamente, un trigger se activa solo cuando ocurre el evento al que está asociado.

El trigger `generar_match` es el componente más importante de la lógica de negocio de LoveCode. Es el responsable de detectar cuando dos usuarios se han dado like mutuamente y crear automáticamente el match entre ellos. Su funcionamiento es el siguiente:

1. El trigger está asociado al evento `AFTER INSERT ON Likes`, lo que significa que se ejecuta automáticamente cada vez que se inserta un nuevo registro en la tabla `Likes`, es decir, cada vez que un usuario da like a otro.
2. Cuando se activa, el trigger comprueba si ya existe un registro en la tabla `Likes` con el emisor y el receptor invertidos. Es decir, comprueba si el usuario que acaba de recibir el like ya había dado like previamente al usuario que lo ha enviado.
3. Si existe ese like recíproco, significa que hay un match. En ese caso, el trigger inserta automáticamente un nuevo registro en la tabla `Matches` con los identificadores de los dos usuarios. Se utiliza `INSERT IGNORE` para evitar crear un match duplicado en caso de que el trigger se ejecute en situaciones concurrentes.

```
DELIMITER $$

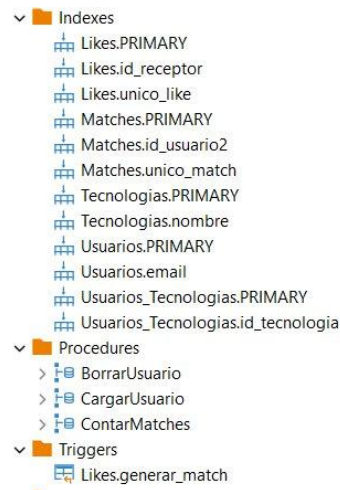
▷Run | ☐Select
CREATE TRIGGER generar_match
AFTER INSERT ON Likes
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1 FROM Likes
        WHERE id_emisor = NEW.id_receptor
        AND id_receptor = NEW.id_emisor
    ) THEN
        INSERT IGNORE INTO Matches (id_usuario1, id_usuario2)
        VALUES (
            LEAST(NEW.id_emisor, NEW.id_receptor),
            GREATEST(NEW.id_emisor, NEW.id_receptor)
        );
    END IF;
END$$

▷Run
DELIMITER ;
```

La importancia de este trigger radica en que automatiza completamente la lógica de detección de matches dentro de la propia base de datos, sin necesidad de que el backend tenga que hacer

comprobaciones adicionales. Es un ejemplo claro de cómo la base de datos puede encargarse de parte de la lógica del negocio de forma eficiente.

En el panel de objetos de DBeaver se puede verificar que los tres procedimientos almacenados y el trigger están correctamente registrados y disponibles en la base de datos:



3.7 Inserción de datos de prueba

Para poder probar que el sistema funciona correctamente desde el principio, es necesario disponer de datos de prueba en todas las tablas. Estos datos simulan usuarios reales con sus tecnologías, likes y matches, y permiten verificar que las consultas, procedimientos y el trigger funcionan como se espera.

Se han insertado diez tecnologías que cubren las más utilizadas en el desarrollo web y en los sistemas informáticos:

```
INSERT INTO Tecnologias (nombre) VALUES
('Java'), ('Python'), ('JavaScript'), ('SQL'), ('HTML'),
('CSS'), ('Spring Boot'), ('React'), ('Git'), ('Linux');
```

A continuación se han creado cinco usuarios de prueba con perfiles variados que representan diferentes tipos de desarrolladores:

```
INSERT INTO Usuarios (nombre, email, password, descripcion) VALUES
('Carlos Lopez', 'carlos@lovecode.com', '1234', 'Frontend lover, React y CSS.'),
('Ana Garcia', 'ana@lovecode.com', '1234', 'Backend con Java y Spring Boot.'),
('Laura Martinez', 'laura@lovecode.com', '1234', 'Fullstack, Python y JavaScript.'),
('David Torres', 'david@lovecode.com', '1234', 'DBA, SQL y Linux.'),
('Marta Ruiz', 'marta@lovecode.com', '1234', 'DevOps, Git y automatizacion.');
```

Después se han asignado tecnologías a cada usuario mediante la tabla intermedia Usuarios_Tecnologías, de forma que cada usuario tenga entre tres y cuatro tecnologías asociadas:


```
INSERT INTO Usuarios_Tecnologias (id_usuario, id_tecnologia) VALUES
  (1,3),(1,5),(1,6),(1,8),
  (2,1),(2,4),(2,7),(2,9),
  (3,2),(3,3),(3,4),(3,9),
  (4,4),(4,10),(4,1),
  (5,9),(5,10),(5,3);
```

Por último se han insertado likes entre usuarios, incluyendo deliberadamente likes en ambas direcciones entre algunos pares de usuarios para que el trigger generar_match se active y cree matches automáticamente:

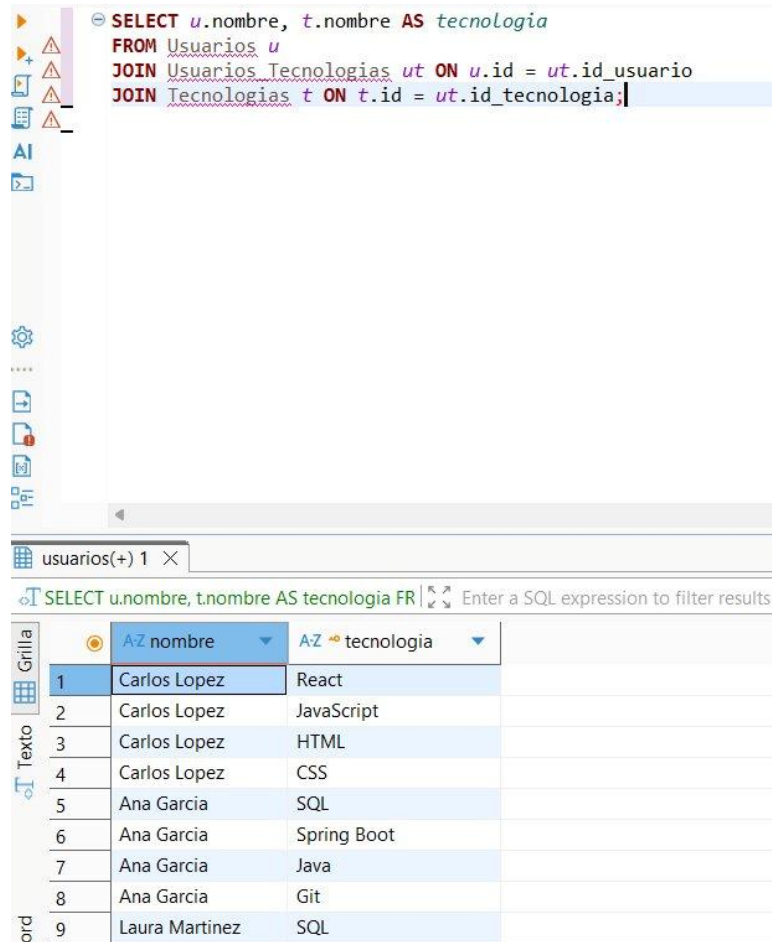
```
INSERT INTO Likes (id_emisor, id_receptor) VALUES (1, 2);
▷Run
INSERT INTO Likes (id_emisor, id_receptor) VALUES (2, 1);
▷Run
INSERT INTO Likes (id_emisor, id_receptor) VALUES (3, 1);
▷Run
INSERT INTO Likes (id_emisor, id_receptor) VALUES (4, 5);
▷Run
INSERT INTO Likes (id_emisor, id_receptor) VALUES (5, 4);
```

3.8 Consultas SQL desarrolladas

Las consultas SQL son las instrucciones que permiten extraer información de la base de datos de formas muy variadas: filtrando, ordenando, agrupando o combinando datos de varias tablas. Se han desarrollado ocho consultas que cubren diferentes tipos de operaciones, desde las más básicas hasta las más avanzadas.

Consulta 1 — JOIN: usuarios con sus tecnologías

Esta consulta combina tres tablas (Usuarios, Usuarios_Tecnologías y Tecnologías) mediante dos operaciones JOIN para mostrar qué tecnología conoce cada usuario. Es un ejemplo de JOIN múltiple, que es la operación más habitual cuando se trabaja con bases de datos relacionales. El resultado muestra una fila por cada combinación usuario-tecnología.



```

SELECT u.nombre, t.nombre AS tecnologia
FROM Usuarios u
JOIN Usuarios_Tecnologias ut ON u.id = ut.id_usuario
JOIN Tecnologias t ON t.id = ut.id_tecnologia;

```

usuarios(+) 1 X

SELECT u.nombre, t.nombre AS tecnologia FR | Enter a SQL expression to filter results

	AZ nombre	AZ tecnologia
1	Carlos Lopez	React
2	Carlos Lopez	JavaScript
3	Carlos Lopez	HTML
4	Carlos Lopez	CSS
5	Ana Garcia	SQL
6	Ana Garcia	Spring Boot
7	Ana Garcia	Java
8	Ana Garcia	Git
9	Laura Martinez	SQL

Consulta 2 — COUNT + GROUP BY: número de tecnologías por usuario

Esta consulta utiliza la función de agregación COUNT junto con la cláusula GROUP BY para calcular cuántas tecnologías tiene asignadas cada usuario. El GROUP BY agrupa todas las filas que tienen el mismo usuario y el COUNT cuenta cuántas hay en cada grupo. Es una de las operaciones estadísticas más comunes en SQL.


```
SELECT u.nombre, COUNT(ut.id_tecnologia) AS num_tecnologias
FROM Usuarios u
LEFT JOIN Usuarios_Tecnologias ut ON u.id = ut.id_usuario
GROUP BY u.id, u.nombre;
```

1	×	
u.nombre, COUNT(ut.id_tecnologia) Enter a SQL expression to filter results (use Ct		
A-Z nombre	123 num_tecnologias	
Carlos Lopez	4	
Ana Garcia	4	
Laura Martinez	4	
David Torres	3	
Marta Ruiz	3	
carlos	3	

Consulta 3 — ORDER BY: usuarios por fecha de registro

Esta consulta obtiene el listado de usuarios con su nombre, email y fecha de registro, ordenados del más reciente al más antiguo mediante la cláusula ORDER BY con la opción DESC (descendente). Esta consulta es útil, por ejemplo, para ver los últimos usuarios que se han unido a la plataforma.

```
SELECT nombre, email, fecha_registro
FROM Usuarios
ORDER BY fecha_registro DESC;
```

arios 1 X

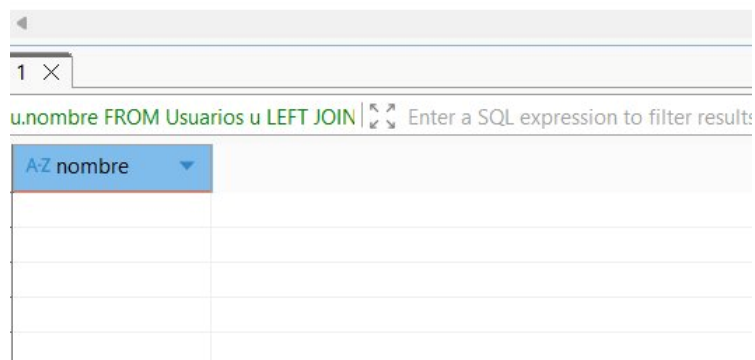
ECT nombre, email, fecha_registro FROM I | Enter a SQL expression to filter resul

A-Z nombre	A-Z email	fecha_registro
Carlos Lopez	carlos@lovecode.com	2026-05-29 18:08:12
Ana Garcia	ana@lovecode.com	2026-05-29 18:08:12
Laura Martinez	laura@lovecode.com	2026-05-29 18:08:12
David Torres	david@lovecode.com	2026-05-29 18:08:12
Marta Ruiz	marta@lovecode.com	2026-05-29 18:08:12

Consulta 4 — LEFT JOIN: usuarios sin tecnologías asignadas

Esta consulta utiliza un LEFT JOIN para encontrar usuarios que no tienen ninguna tecnología asignada. El LEFT JOIN incluye todos los registros de la tabla Usuarios independientemente de si tienen correspondencia en Usuarios_Tecnologías o no. El filtro WHERE ut.id_tecnologia IS NULL selecciona únicamente aquellos usuarios para los que no existe ningún registro en Usuarios_Tecnologías, es decir, los que no tienen tecnologías.

```
SELECT u.nombre
FROM Usuarios u
LEFT JOIN Usuarios_Tecnologias ut ON u.id = ut.id_usuario
WHERE ut.id_tecnologia IS NULL;
```



The screenshot shows a database query interface. At the top, there is a query editor with the text: `u.nombre FROM Usuarios u LEFT JOIN`. Below the query editor, there is a button labeled "AZ nombre" with a dropdown arrow. Below the button, there is a table with 5 empty rows and 2 columns.

AZ nombre	

Consulta 5 — Matches con nombres de ambos usuarios

Esta consulta obtiene todos los matches registrados en el sistema, pero en lugar de mostrar los identificadores numéricos de los usuarios, muestra sus nombres reales. Para conseguirlo, realiza un doble JOIN con la tabla Usuarios usando dos alias diferentes (u1 y u2), uno para el primer usuario del match y otro para el segundo.

```

SELECT u1.nombre AS usuario1, u2.nombre AS usuario2, m.fecha
FROM Matches m
JOIN Usuarios u1 ON m.id_usuario1 = u1.id
JOIN Usuarios u2 ON m.id_usuario2 = u2.id;

```

Usuarios(+)			
SELECT u1.nombre AS usuario1, u2.nombre AS usuario2, m.fecha			
A-Z usuario1	A-Z usuario2	fecha	
Carlos Lopez	Ana Garcia	2026-05-29 18:10:37	
David Torres	Marta Ruiz	2026-05-29 18:10:50	

Consulta 6 — Ranking de tecnologías más populares

Esta consulta calcula cuántos usuarios tienen asignada cada tecnología y presenta el resultado ordenado de mayor a menor popularidad. Combina un JOIN entre Tecnologías y Usuarios_Tecnologías con la función COUNT y las cláusulas GROUP BY y ORDER BY. Es muy útil para conocer qué tecnologías tienen más demanda en la comunidad de LoveCode.

```

SELECT t.nombre, COUNT(ut.id_usuario) AS num_usuarios
FROM Tecnologias t
JOIN Usuarios_Tecnologias ut ON t.id = ut.id_tecnologia
GROUP BY t.id, t.nombre
ORDER BY num_usuarios DESC;

```

Tecnologías 1 X	
SELECT t.nombre, COUNT(ut.id_usuario) AS num_usuarios	
A-Z nombre	123 num_usuarios
Git	4
JavaScript	3
Linux	3
SQL	3
CSS	2
Java	2
HTML	1
Python	1
React	1
Spring Boot	1

Consulta 7 — Likes enviados por usuario

Esta consulta cuenta cuántos likes ha enviado cada usuario del sistema. Utiliza un LEFT JOIN para incluir también a los usuarios que no han enviado ningún like, que aparecerán con el valor 0 en lugar de quedar excluidos del resultado. Esto permite obtener una visión completa de la actividad de todos los usuarios.

```

SELECT u.nombre, COUNT(l.id) AS likes_dados
FROM Usuarios u
LEFT JOIN Likes l ON u.id = l.id_emisor
GROUP BY u.id, u.nombre;

```

Usuarios 1 X	
SELECT u.nombre, COUNT(l.id) AS likes_dados Enter a SQL expression to filter results	
AZ nombre	likes_dados
Carlos Lopez	1
Ana Garcia	1
Laura Martinez	1
David Torres	1
Marta Ruiz	1
carlos	5

Consulta 8 — Vista vista_perfiles

Una vista es una consulta SQL guardada con un nombre que se puede utilizar como si fuera una tabla más de la base de datos. La vista vista_perfiles combina la información de Usuarios, Usuarios_Tecnologías y Tecnologías para devolver, en una sola consulta, el identificador, nombre, descripción y la lista de tecnologías de cada usuario agrupadas en una sola celda. Esta vista es especialmente importante para el backend, que la utiliza directamente para cargar los perfiles de la página principal.

```
SELECT * FROM vista_perfiles;
```

(+)			
* FROM vista_perfiles			
Enter a SQL expression to filter results (use Ctrl+Space)			
id	nombre	descripcion	tecnologias
1	Carlos Lopez	Frontend lover, React y CSS.	JavaScript, HTML, CSS, React
2	Ana Garcia	Backend con Java y Spring Boot.	Java, SQL, Spring Boot, Git
3	Laura Martinez	Fullstack, Python y JavaScript.	Python, JavaScript, SQL, Git
4	David Torres	DBA, SQL y Linux.	Java, SQL, Linux
5	Marta Ruiz	DevOps, Git y automatizacion.	JavaScript, Git, Linux
6	carlos	hola	CSS, Git, Linux

4. Programación (Java con Spring Boot)

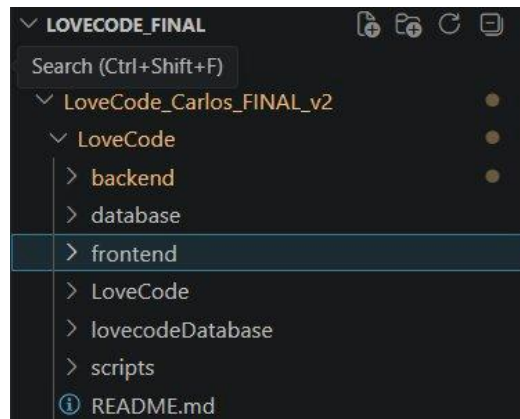
El módulo de Programación cubre el desarrollo completo del servidor backend de LoveCode. El backend es la parte de la aplicación que no ve directamente el usuario, pero que es responsable de que todo el sistema funcione correctamente: recibe las peticiones que llegan desde el navegador, aplica la lógica del negocio, consulta o modifica la base de datos y devuelve los resultados.

Para el desarrollo del backend se ha utilizado Java, uno de los lenguajes de programación más extendidos en el entorno empresarial, junto con Spring Boot, un framework que facilita enormemente la creación de aplicaciones web y APIs en Java. Spring Boot proporciona configuraciones predeterminadas inteligentes, un servidor web embebido y una estructura de proyecto bien definida, lo que permite centrarse en desarrollar la lógica de la aplicación en lugar de perder tiempo en configuraciones.

4.1 Estructura del proyecto

El proyecto Java sigue una arquitectura en capas, que es el patrón de organización más utilizado en el desarrollo profesional de aplicaciones web. Separar el código en capas con responsabilidades bien definidas facilita el mantenimiento, las pruebas y la evolución del sistema:

- **Controladores (Controllers):** son la puerta de entrada al servidor. Cada controlador define un conjunto de endpoints (direcciones URL) a los que el navegador puede enviar peticiones. El controlador recibe la petición, extrae los datos que necesita y los pasa al servicio correspondiente.
- **Servicios (Services):** contienen la lógica del negocio. Son los responsables de coordinar las operaciones, validar los datos y decidir qué hacer en cada situación.
- **Repositorios (Repositories):** se comunican directamente con la base de datos usando JPA (Java Persistence API). Proporcionan métodos para buscar, guardar, actualizar y eliminar registros.
- **Modelos (Models):** son clases Java que representan las entidades de la base de datos (Usuario, Tecnología, Like, Match). JPA se encarga de mapear automáticamente estos objetos con las tablas correspondientes.



4.2 Conexión con la base de datos

La conexión entre el servidor Java y la base de datos MySQL se gestiona mediante JPA con Hibernate como implementación subyacente. JPA es una especificación de Java que define cómo trabajar con bases de datos relacionales usando objetos Java, evitando en muchos casos la necesidad de escribir SQL directamente. Hibernate es la librería más popular que implementa esta especificación.

La configuración de la conexión se define en el archivo `application.properties` del proyecto Spring Boot, donde se especifica la URL completa de la base de datos, el usuario y la contraseña. Al arrancar la aplicación, Spring Boot lee esta configuración, establece el pool de conexiones y verifica que la base de datos es accesible.

En la siguiente captura se puede observar la consola de arranque del servidor, donde se confirma que la conexión se establece correctamente, se muestra el listado de usuarios registrados en la base de datos a modo de verificación y se indica que el servidor ha quedado escuchando en el puerto 8080:

```
PS C:\Users\carlo\Documents\LoveCode_Final\LoveCode_Carlos_FINAL_v2\LoveCode\backend> mvn spring-boot
:run
2026-05-30T12:56:10.256+02:00 INFO 12872 --- [          main] j.LocalContainerEntityManagerFactoryB
ean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-05-30T12:56:10.480+02:00 INFO 12872 --- [          main] o.s.d.j.r.query.QueryEnhancerFactory
: Hibernate is in classpath; If applicable, HQL parser will be used.
2026-05-30T12:56:10.727+02:00 WARN 12872 --- [          main] JpaBaseConfiguration$JpaWebConfigurat
ion : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed dur
ing view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-05-30T12:56:10.947+02:00 INFO 12872 --- [          main] o.s.b.w.embedded.tomcat.TomcatWebServ
er : Tomcat started on port 8080 (http) with context path ''
2026-05-30T12:56:10.952+02:00 INFO 12872 --- [          main] com.example.Main
: Started Main in 3.326 seconds (process running for 3.595)
Conexión exitosa a LoveCode: 1
Usuarios registrados:
- Carlos Lopez (carlos@lovecode.com)
- Ana Garcia (ana@lovecode.com)
- Laura Martinez (laura@lovecode.com)
- David Torres (david@lovecode.com)
- Marta Ruiz (marta@lovecode.com)
- carlos (angellovecode@gmail.com)
Servidor LoveCode arrancado en http://localhost:8080
```

4.3 Endpoints de la API REST

Los endpoints son las "direcciones" del servidor a las que el navegador puede enviar peticiones HTTP. Cada endpoint está asociado a una URL específica y a un método HTTP (GET para obtener datos, POST para enviar datos, PUT para actualizar, DELETE para eliminar). El conjunto de todos los endpoints forma lo que se conoce como una API REST (Application Programming Interface de tipo REpresentational State Transfer).

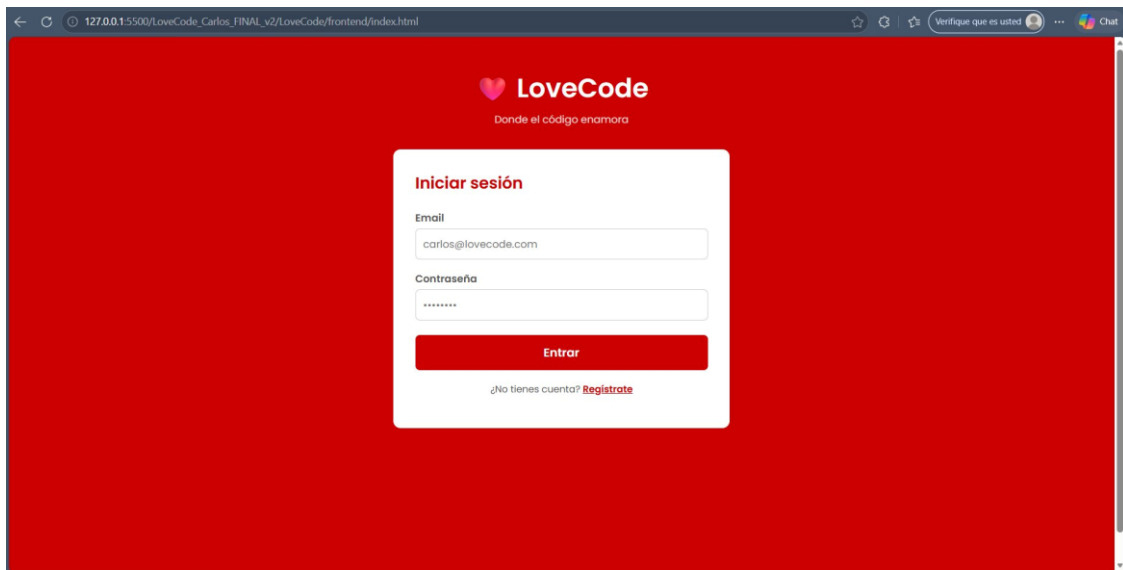
Los endpoints implementados en LoveCode son los siguientes:

- POST /api/registro: recibe los datos de un nuevo usuario (nombre, email, contraseña, descripción y lista de tecnologías) y crea su perfil en la base de datos. Si el email ya existe, devuelve un mensaje de error.
- POST /api/login: recibe el email y la contraseña de un usuario y verifica si las credenciales son correctas consultando la base de datos. Si son correctas, devuelve una respuesta positiva que permite al frontend redirigir al usuario a la página de perfiles.
- GET /api/perfiles: devuelve la lista completa de todos los usuarios registrados con su información de perfil y sus tecnologías, obtenida mediante la vista vista_perfiles de la base de datos.
- POST /api/like: recibe el identificador del usuario que da el like y el del usuario que lo recibe, registra el like en la base de datos y devuelve si se ha producido un match.
- GET /api/match/xml: genera y devuelve la ficha de compatibilidad en formato HTML cuando se produce un match entre dos usuarios.

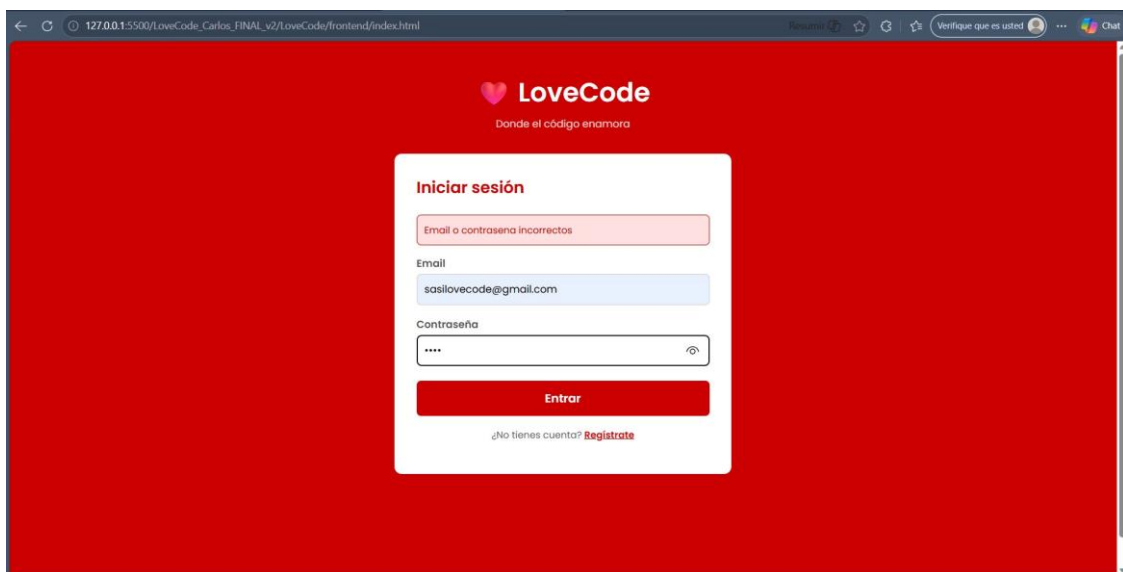
4.4 Pantalla de inicio de sesión

La página de inicio de sesión es la primera que ve el usuario al acceder a LoveCode. Presenta el logotipo de la aplicación y un formulario con dos campos: el email y la contraseña. Al pulsar el botón "Entrar", el código JavaScript de la página recoge los valores de los campos y los envía al servidor mediante una petición POST asíncrona al endpoint /api/login.

El servidor valida las credenciales comprobando si existe un usuario con ese email y si la contraseña coincide. Si las credenciales son correctas, el frontend redirige automáticamente al usuario a la página de perfiles. Si no, se muestra un mensaje de error:



Es importante destacar que el mensaje de error dice "Email o contraseña incorrectos" sin especificar cuál de los dos datos es el que falla. Esto es una práctica de seguridad deliberada: si el sistema especificara cuál de los dos es incorrecto, estaría dando información a posibles atacantes sobre si un email concreto está o no registrado en el sistema.



4.5 Registro de nuevos usuarios

La página de registro permite a cualquier persona crear una cuenta en LoveCode. El formulario incluye los campos de nombre completo, correo electrónico, contraseña y una descripción libre del perfil. Además, incluye una sección de selección de tecnologías mediante checkboxes.

Una característica importante es que las tecnologías no están escritas directamente en el HTML sino que se cargan dinámicamente desde la API al abrir la página. Esto significa que si en el futuro se añade una nueva tecnología a la base de datos, aparecerá automáticamente en el formulario de registro sin necesidad de modificar ningún archivo HTML.

Crea tu perfil de desarrollador

Registro

Nombre
Tu nombre

Email
tu@correo.com

Contraseña
.....

Descripción
Cuéntanos algo sobre ti..

Tecnologías que dominas

☐ CSS	☐ Git
☐ HTML	☐ Java
☐ JavaScript	☐ Linux
☐ Python	☐ React
☐ Spring Boot	☐ SQL

Crear cuenta

Al completar el formulario y pulsar "Crear cuenta", JavaScript recoge todos los datos, incluyendo los identificadores de las tecnologías seleccionadas, y los envía al servidor en formato JSON. El servidor crea el usuario en la tabla Usuarios y registra sus tecnologías en Usuarios_Tecnologías:

Crea tu perfil de desarrollador

Registro

Nombre
sasi

Email
sasilovecode@gmail.com

Contraseña
....

Descripción
soy del psoe

Tecnologías que dominas

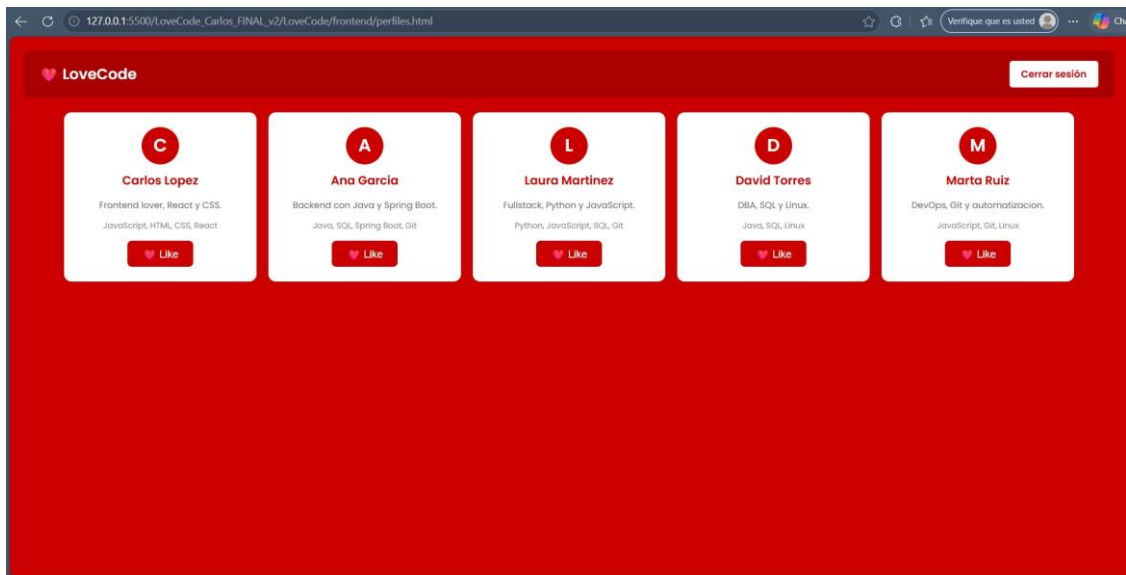
☐ CSS	☐ Git
☒ HTML	☐ Java
☒ JavaScript	☐ Linux
☐ Python	☒ React
☐ Spring Boot	☐ SQL

Crear cuenta

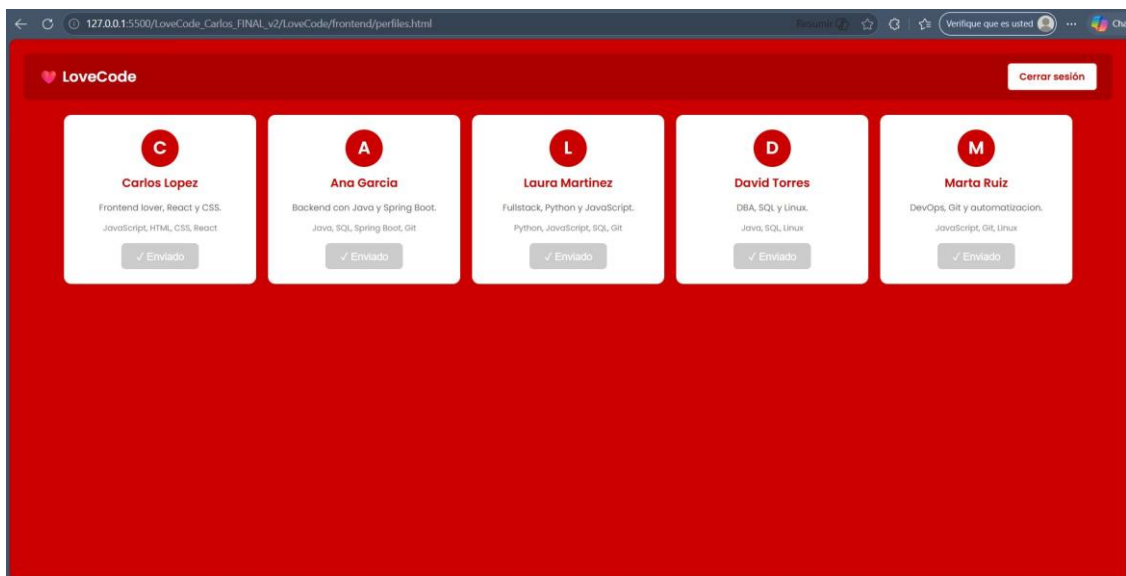
4.6 Vista de perfiles y sistema de likes

Una vez que el usuario ha iniciado sesión, accede a la página principal de la aplicación: la vista de perfiles. Esta página carga automáticamente la lista de todos los desarrolladores registrados y los muestra como tarjetas visuales. Las tarjetas se generan completamente de forma dinámica con JavaScript: se realiza una petición GET al servidor, se recibe la lista de usuarios en formato JSON y para cada usuario se crea el HTML de su tarjeta de perfil de forma programática.

Cada tarjeta muestra el avatar con la inicial del nombre del desarrollador, su nombre completo, su descripción y la lista de tecnologías que domina. Al pie de cada tarjeta aparece el botón de Like:

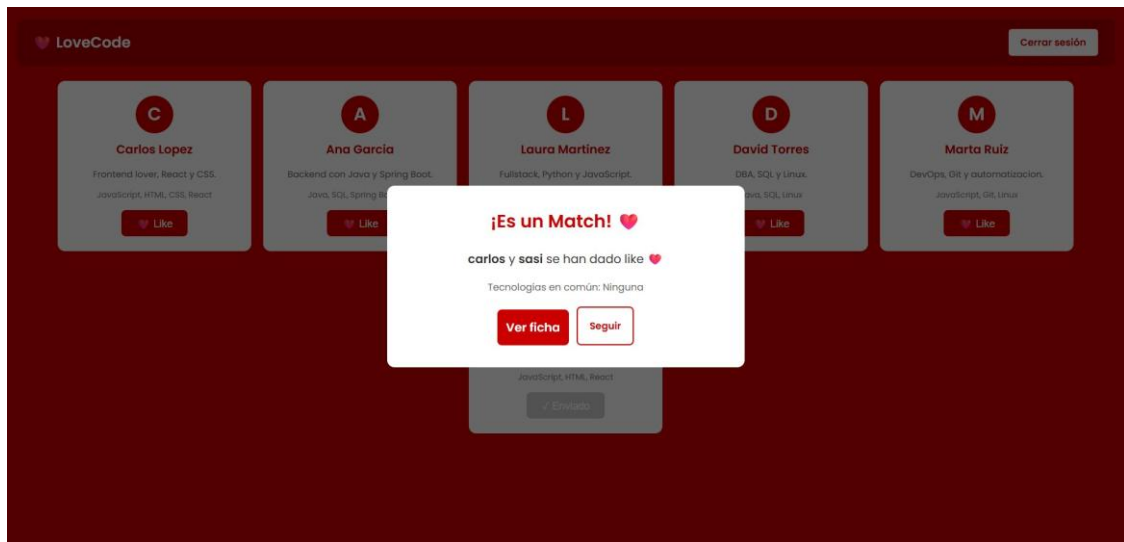


Al pulsar el botón de Like en una tarjeta, JavaScript envía una petición POST al servidor con el identificador del usuario que da el like y el del usuario que lo recibe. Una vez enviado, el botón cambia de estado visual a "Enviado" con un color gris para indicar que el like ya ha sido registrado y no se puede dar dos veces al mismo perfil:

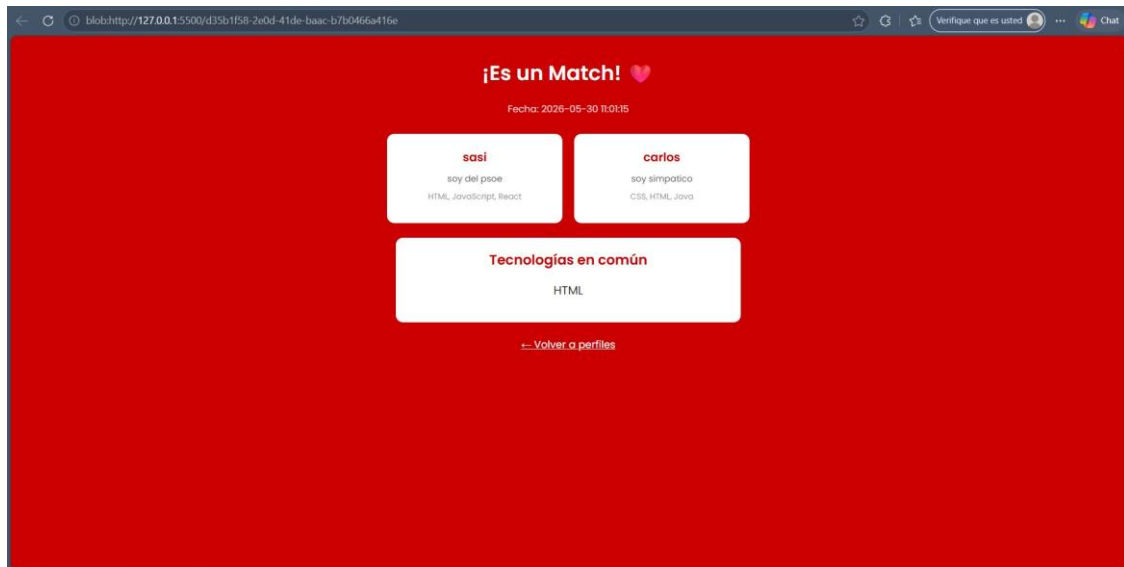


4.7 Detección del match y ficha de compatibilidad

Cuando el servidor procesa un like y detecta que el usuario receptor ya había dado like al emisor previamente, determina que se ha producido un match. En ese caso, el servidor devuelve al frontend una respuesta especial indicando que ha habido coincidencia. JavaScript recibe esta respuesta y muestra automáticamente un popup de notificación sobre la página de perfiles:



El popup muestra los nombres de los dos usuarios que han hecho match y las tecnologías que tienen en común. Desde el popup, el usuario puede acceder a la ficha completa de compatibilidad, que presenta los perfiles de ambos desarrolladores lado a lado con toda su información y las tecnologías compartidas resaltadas:



5. Lenguajes de Marcas (HTML, CSS y JavaScript)

El módulo de Lenguajes de Marcas cubre el desarrollo de la interfaz web de LoveCode, es decir, todo lo que el usuario percibe y con lo que interactúa directamente en su navegador. Este módulo abarca tres tecnologías que trabajan de forma complementaria: HTML5 para definir la estructura y el contenido de las páginas, CSS3 para aplicar el diseño visual y la apariencia, y JavaScript para añadir comportamiento dinámico y gestionar la comunicación con el servidor.

5.1 Identidad visual y diseño de la interfaz

La interfaz de LoveCode sigue una identidad visual coherente y reconocible en todas las páginas. El diseño se basa en una paleta de colores que combina el rojo intenso como color principal, que evoca tanto la pasión como la energía tecnológica, con el blanco para el contenido y el gris oscuro para los textos secundarios.

Las páginas utilizan un fondo de color rojo oscuro que crea un contraste elegante con las tarjetas y los formularios de fondo blanco. Los botones principales siguen el mismo color rojo de la marca y se destacan visualmente para guiar al usuario hacia las acciones principales. La tipografía es limpia y legible, sin elementos decorativos que distraigan de la funcionalidad.

El diseño es responsivo, es decir, se adapta a diferentes tamaños de pantalla, aunque en este proyecto está optimizado principalmente para su uso en pantalla de escritorio.

5.2 Página de inicio de sesión (index.html)

La página de inicio de sesión es la entrada principal a la aplicación. Su diseño es minimalista y directo: presenta el logotipo de LoveCode en la parte superior y un formulario centrado con los campos de email y contraseña. El JavaScript de esta página se encarga de gestionar el envío del formulario, interceptar la respuesta del servidor, mostrar un mensaje de error si las credenciales son incorrectas y redirigir al usuario a la página de perfiles si el login es exitoso.

5.3 Página de registro (registro.html)

La página de registro permite a los nuevos usuarios crear su perfil en LoveCode. Lo más destacable de esta página es que las tecnologías disponibles no están escritas directamente en el HTML, sino que se cargan dinámicamente desde la API del servidor al abrir la página. JavaScript hace una petición GET al servidor, recibe la lista de tecnologías y genera los checkboxes de forma automática. Esto hace que la página esté siempre sincronizada con el catálogo de tecnologías de la base de datos.

5.4 Página de perfiles (perfiles.html)

Esta es la página principal de la aplicación una vez que el usuario ha iniciado sesión. Toda su funcionalidad depende de JavaScript: al cargar la página, JavaScript hace una petición GET al servidor para obtener la lista de perfiles, procesa la respuesta JSON y genera dinámicamente el HTML de cada tarjeta. También gestiona el sistema de likes (detectando qué usuarios ya han recibido like para cambiar el estado del botón), el popup de match y la navegación a la ficha de compatibilidad.

5.5 Comunicación asíncrona con el backend

Toda la comunicación entre el frontend y el backend de LoveCode se realiza de forma asíncrona mediante la función `fetch()` de JavaScript. Esta función permite enviar peticiones HTTP al servidor sin interrumpir la ejecución del resto del código, de forma que la página sigue siendo interactiva mientras espera la respuesta del servidor.

El patrón utilizado es el siguiente: se llama a `fetch()` con la URL del endpoint y los parámetros necesarios (método HTTP, cabeceras y cuerpo de la petición), se espera la respuesta de forma asíncrona con `await`, se procesa la respuesta (generalmente parseando el JSON devuelto) y se actualiza la interfaz de usuario en consecuencia.

Esta forma de trabajar, denominada comunicación asíncrona, es lo que permite que LoveCode funcione como una aplicación web moderna: las páginas no se recargan completamente con cada acción, sino que solo se actualiza la parte de la interfaz que necesita cambiar, ofreciendo una experiencia de usuario mucho más fluida y ágil.

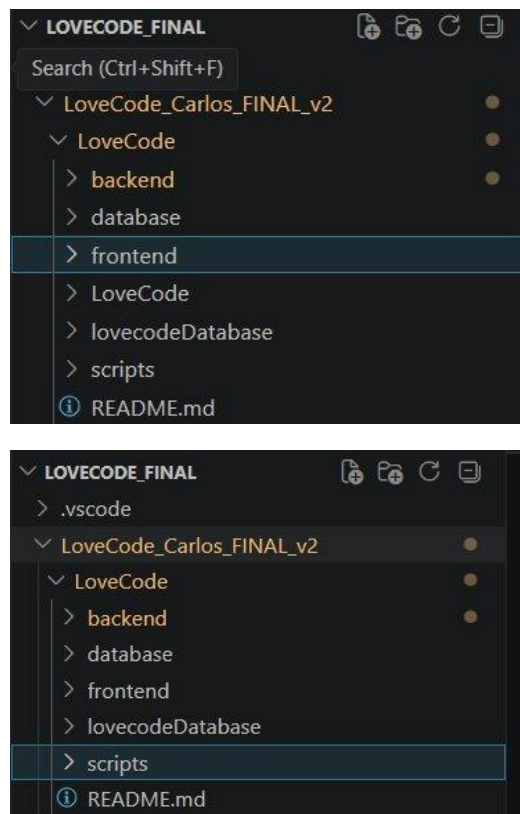
6. Entornos de Desarrollo (Git y GitHub)

El módulo de Entornos de Desarrollo cubre la gestión del código fuente del proyecto mediante sistemas de control de versiones. Un sistema de control de versiones es una herramienta que registra de forma detallada todos los cambios que se realizan en el código a lo largo del tiempo, permitiendo volver a cualquier versión anterior, trabajar en paralelo sobre diferentes funcionalidades sin interferencias y colaborar de forma organizada con otros desarrolladores.

Para LoveCode se ha utilizado Git como sistema de control de versiones local y GitHub como plataforma de alojamiento del repositorio remoto. Git es actualmente el sistema de control de versiones más utilizado en el mundo del desarrollo de software, y GitHub es la plataforma de alojamiento de repositorios Git más popular.

6.1 Estructura del repositorio

El repositorio de LoveCode está organizado en carpetas diferenciadas para separar claramente las distintas partes del proyecto. Esta organización facilita la navegación por el código y permite que cualquier persona que acceda al repositorio pueda entender rápidamente qué contiene cada carpeta:

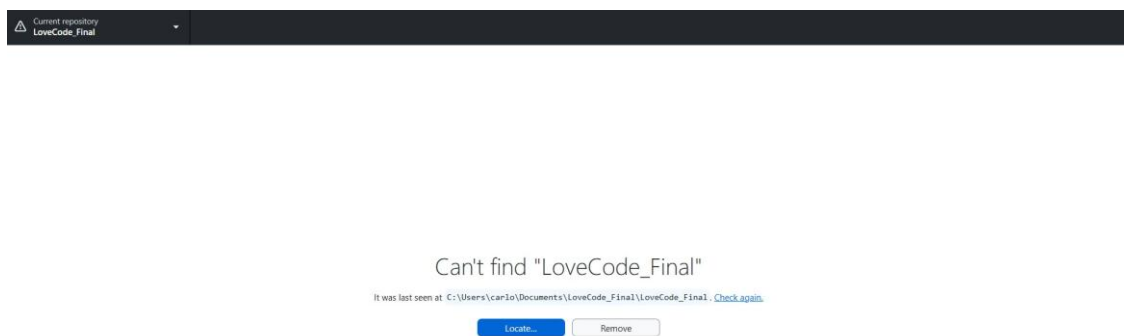


- backend/: contiene todo el código fuente del servidor Java con Spring Boot, incluyendo los controladores, servicios, repositorios y modelos.

- database/: contiene los scripts SQL necesarios para crear la base de datos, sus tablas, procedimientos, trigger y datos de prueba.
- frontend/: contiene las páginas HTML, los archivos CSS de estilos y el código JavaScript de la interfaz web.
- scripts/: contiene los scripts de automatización del sistema, como el script de backup de la base de datos y el script de arranque.
- README.md: archivo de presentación del proyecto que explica qué es LoveCode, qué tecnologías utiliza y cómo instalar y ejecutar el sistema.

6.2 Problema con GitHub Desktop y solución adoptada

Durante el desarrollo se pretendía utilizar GitHub Desktop para gestionar los commits y subir los cambios al repositorio remoto de forma cómoda mediante una interfaz gráfica. Sin embargo, GitHub Desktop mostraba el error "Can't find LoveCode_Final" indicando que no podía localizar la carpeta del repositorio local en la ubicación donde la tenía registrada.



Este problema impedía hacer commits y sincronizar el código local con el repositorio remoto de GitHub a través de GitHub Desktop. Ante esta situación, se buscó una solución alternativa para poder igualmente subir el código al repositorio.

La solución adoptada fue utilizar la interfaz web de GitHub directamente. GitHub permite subir archivos desde el navegador accediendo al repositorio, navegando a la carpeta correspondiente y usando la opción de carga de archivos. Aunque este método no genera un historial de commits tan detallado como el flujo normal de Git, sí permite tener todo el código disponible en el repositorio remoto para que el profesorado pueda revisarlo.

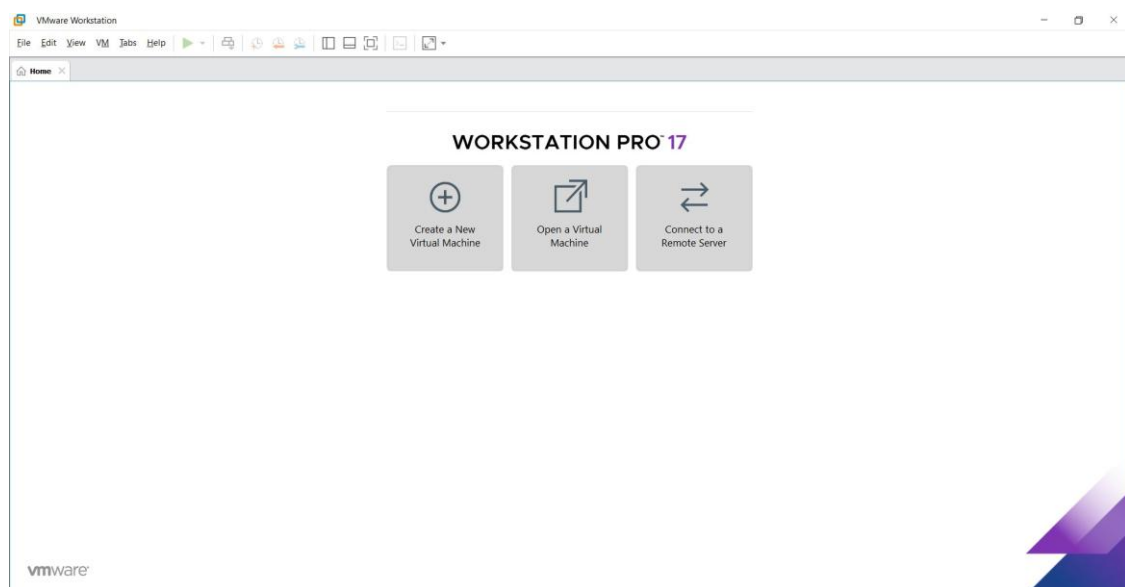
7. Sistemas Informáticos

El módulo de Sistemas Informáticos se encarga de todo lo relacionado con el entorno donde se ejecuta la aplicación: el sistema operativo, el servidor de base de datos, la virtualización, las copias de seguridad y la automatización de tareas del sistema. Son aspectos que a veces pasan desapercibidos para el usuario final pero que son fundamentales para que una aplicación funcione de forma estable y segura.

7.1 De la máquina virtual a Docker

El plan inicial para el módulo de Sistemas Informáticos era utilizar una máquina virtual con el sistema operativo Ubuntu Linux como entorno donde ejecutar el servidor de base de datos MariaDB. Las máquinas virtuales son una tecnología de virtualización que permite ejecutar un sistema operativo completo dentro de otro, creando un entorno aislado y controlado. Para este proyecto se utilizó VMware Workstation Pro 17 como software de virtualización.

Sin embargo, durante el desarrollo se produjo un incidente: la máquina virtual Ubuntu que se había creado y configurado fue eliminada accidentalmente del sistema. Al intentar acceder a ella, VMware Workstation mostraba únicamente la pantalla de inicio sin ninguna máquina virtual disponible, confirmando la pérdida del entorno de base de datos configurado hasta ese momento:



Ante esta situación, fue necesario buscar una alternativa para poder continuar con el desarrollo sin perder más tiempo. La solución elegida fue migrar la base de datos a Docker, una tecnología de contenedores que permite ejecutar aplicaciones de forma aislada sin necesidad de una máquina virtual completa.

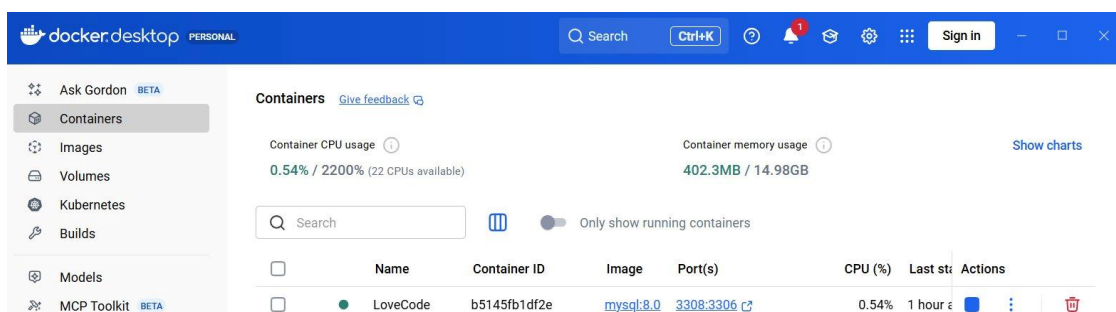
Docker funciona de una manera diferente a las máquinas virtuales. Mientras que una máquina virtual emula un ordenador completo con su propio sistema operativo, un contenedor Docker comparte el

kernel del sistema operativo anfitrión y solo incluye lo estrictamente necesario para ejecutar la aplicación. Esto hace que los contenedores sean mucho más ligeros, rápidos y eficientes que las máquinas virtuales.

Las ventajas concretas de usar Docker para la base de datos de LoveCode son:

- Velocidad de puesta en marcha: levantar el contenedor MySQL con la base de datos tarda apenas unos segundos, mientras que arrancar una máquina virtual completa puede llevar varios minutos.
- Portabilidad total: el contenedor Docker se puede mover a cualquier ordenador que tenga Docker instalado y funcionará exactamente igual, sin necesidad de reconfiguración.
- Aislamiento del sistema: el contenedor está completamente separado del sistema operativo del ordenador, evitando conflictos con otras aplicaciones o servicios instalados.
- Reproducibilidad: con un solo comando se puede recrear el mismo entorno de base de datos exacto en cualquier máquina.
- Menor consumo de recursos: Docker consume significativamente menos memoria RAM y CPU que una máquina virtual completa con su sistema operativo.

El contenedor Docker con MySQL 8.0 para LoveCode se configura en el puerto 3308 del ordenador local. Se usa el puerto 3308 en lugar del estándar 3306 para evitar conflictos con posibles instalaciones de MySQL ya existentes en el sistema. El contenedor se puede gestionar fácilmente desde Docker Desktop:



7.2 Sistema de copias de seguridad

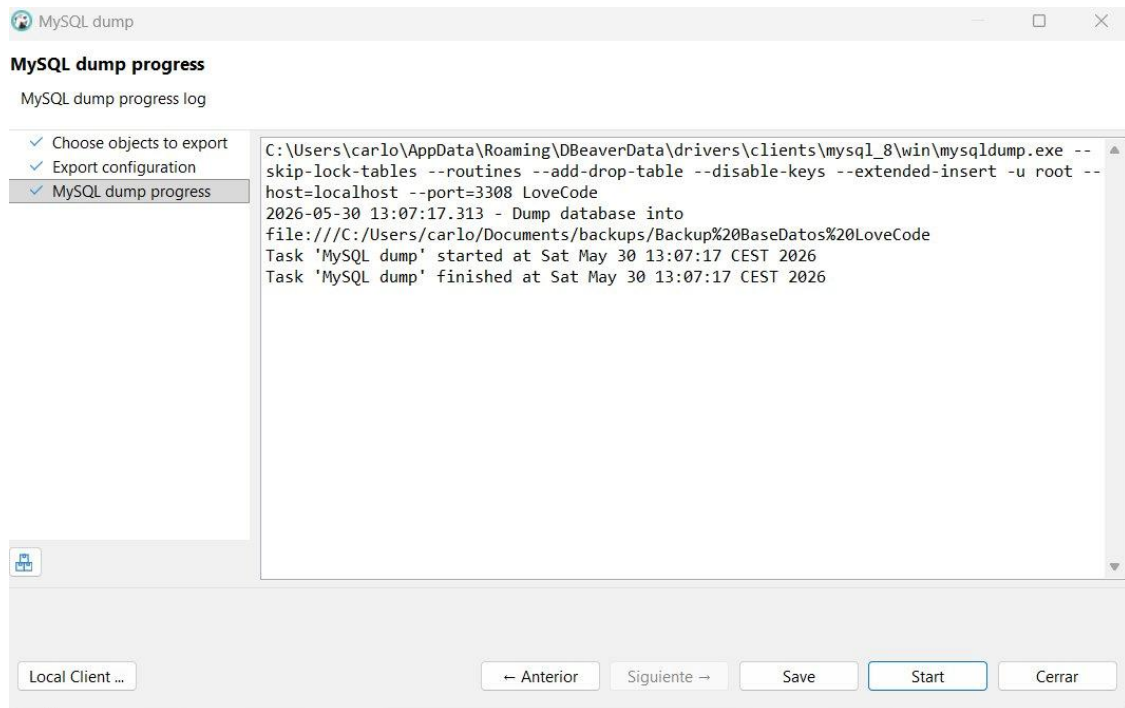
Una copia de seguridad, también llamada backup, es una copia de los datos del sistema que se guarda en un lugar diferente al original, de forma que si los datos originales se pierden o se dañan se puedan recuperar. Realizar copias de seguridad de forma regular es una práctica fundamental en cualquier sistema informático que gestione datos importantes.

Para LoveCode se han implementado dos métodos de backup complementarios:

Backup manual con DBeaver

DBeaver incluye una herramienta integrada de exportación de bases de datos que permite generar un backup con unos pocos clics. Internamente, esta herramienta ejecuta el programa mysqldump,

que es la herramienta oficial de MySQL para exportar bases de datos. El resultado es un archivo de texto con extensión .sql que contiene todas las instrucciones SQL necesarias para recrear la base de datos desde cero: la creación de las tablas, la inserción de todos los datos, los procedimientos almacenados y el trigger.



El archivo de backup se guarda con un nombre que incluye la fecha de generación, lo que facilita identificar cuándo se realizó cada copia y elegir la versión correcta en caso de necesitar restaurar:

 Backup BaseDatos LoveCode	30/05/2026 13:07	Archivo	15 KB
 backup-2026-05-28	28/05/2026 13:32	Archivo de origen ...	0 KB

Script de backup automático (.bat)

Depender únicamente del backup manual tiene un riesgo importante: si se olvida hacerlo, los datos no están protegidos. Para solucionar esto se ha creado un script de comandos de Windows (archivo .bat) que automatiza el proceso de backup. El script ejecuta mysqldump a través del contenedor Docker y guarda el archivo resultante en la carpeta de backups con la fecha actual en el nombre:

```
@echo off
set FECHA=%date:~6,4%-%date:~3,2%-%date:~0,2%
docker exec LoveCode mysqldump -u root -p1234 LoveCode > "%USERPROFILE%\Documents\backups\backup-%FECHA%.sql"
```

7.3 Backup avanzado con volúmenes Docker

Además del script de backup basado en mysqldump, se ha implementado un método alternativo de backup que trabaja directamente con los volúmenes de Docker. Un volumen Docker es el lugar donde el contenedor almacena sus datos de forma persistente. Este método crea una copia comprimida del volumen completo en formato .tar.gz, lo que reduce significativamente el tamaño del archivo de backup y es independiente del estado interno de la base de datos:

```
C:\Users\carlo>docker run --rm ^
¿Más? -v nombre_volumen:/data ^
¿Más? -v %cd%:/backup ^
¿Más? alpine tar czf /backup/backup_%date:~-4%%date:~3,2%%date:~0,2%.tar.gz -C /data .|

C:\Users\carlo>:: Guarda esto como backup_docker.bat
C:\Users\carlo>@echo off
SET VOL=nombre_volumen
SET DEST=C:\Backups\Docker
SET FECHA=%date:~-4%%date:~3,2%%date:~0,2%

if not exist %DEST% mkdir %DEST%

docker run --rm -v %VOL%:/data -v %DEST%:/backup ^
¿Más? alpine tar czf /backup/backup_%FECHA%.tar.gz -C /data .
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
6a0ac1617861: Pull complete
4e0cb4ac63e7: Download complete
a63813f19ccb: Download complete
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest

:: (Opcional) Borrar backups de más de 7 días
forfiles /p "%DEST%" /m *.tar.gz /d -7 /c "cmd /c del @path"
ERROR: No se ha encontrado ningún archivo con el criterio de búsqueda especificado.

echo Backup completado: %DEST%\backup_%FECHA%.tar.gz|
```

7.4 Automatización mediante tareas programadas

Para que el backup se realice automáticamente sin ninguna intervención humana, se ha configurado una tarea programada en el sistema operativo Windows utilizando el comando schtasks. Esta tarea ejecuta el script de backup todos los días a las 2:00 de la madrugada, hora en la que el sistema tiene la menor carga de trabajo y la posibilidad de que alguien esté usando la aplicación es mínima:

```
C:\Users\carlo>schtasks /create /tn "DockerBackup"
¿Más? /tr "C:\ruta\backup_docker.bat" ^
¿Más? /sc daily /st 02:00 /ru SYSTEM|
```

La carpeta de destino del backup se crea automáticamente si no existe y acumula todos los archivos generados, permitiendo recuperar cualquier versión anterior de la base de datos en caso de necesidad:

8. Problemas Encontrados y Soluciones Adoptadas

Durante el desarrollo de cualquier proyecto tecnológico real es habitual encontrarse con dificultades técnicas que no estaban previstas. La capacidad de identificar, analizar y resolver estos problemas de forma metódica es una de las competencias más valoradas en el ámbito del desarrollo de software. En este apartado se recogen los principales obstáculos que surgieron durante el desarrollo de LoveCode y las soluciones que se adoptaron en cada caso.

8.1 Pérdida de la máquina virtual Ubuntu

El primer y más grave problema fue la pérdida accidental de la máquina virtual Ubuntu que alojaba la base de datos de desarrollo. Este incidente obligó a replantear el entorno de trabajo y buscar una alternativa.

Como se explicó en el apartado de Sistemas Informáticos, la solución adoptada fue migrar a Docker. La rapidez con la que se pudo levantar un nuevo entorno de base de datos con Docker (en cuestión de minutos, frente a las horas que habría llevado reinstalar y configurar una nueva máquina virtual) demostró que, en muchos casos, los problemas técnicos también son oportunidades para descubrir soluciones mejores que la original.

8.2 Errores de dependencias en el archivo pom.xml

Al intentar arrancar el servidor Spring Boot por primera vez con el comando "mvn spring-boot:run", el proceso de compilación fallaba con múltiples errores relacionados con las dependencias definidas en el archivo pom.xml. El archivo pom.xml es el archivo de configuración del gestor de dependencias Maven, que es el que se encarga de descargar y gestionar todas las librerías que necesita el proyecto Java.

Los errores se producían principalmente por incompatibilidades entre versiones: la versión de Spring Boot seleccionada no era compatible con la versión del driver de conexión MySQL que se estaba usando, y la versión de JPA/Hibernate tampoco era compatible con el resto de dependencias. Maven no podía resolver estas incompatibilidades automáticamente y el proceso de compilación terminaba con error.

Para resolver el problema se fue revisando el archivo pom.xml de forma sistemática, consultando la documentación oficial de Spring Boot para cada versión y ajustando las versiones de las dependencias hasta encontrar una combinación totalmente compatible. Se utilizó la herramienta en línea Spring Initializr (disponible en start.spring.io) como referencia para obtener las versiones recomendadas de cada librería según la versión de Spring Boot utilizada. Una vez corregidas todas las incompatibilidades, el servidor arrancó correctamente como se puede ver en la captura del arranque del servidor en el apartado de Programación.

8.3 Errores en las consultas SQL por nombres de columna incorrectos

Al ejecutar las consultas SQL en DBeaver, varias de ellas generaban el error "Unknown column", que indica que se está intentando usar una columna que no existe en la tabla especificada. El problema se debía a que los nombres de columna utilizados en los scripts no coincidían con los nombres reales de las columnas en la base de datos del proyecto.

Concretamente, en las tablas Usuarios y Tecnologías la clave primaria se llama simplemente "id" en lugar de "id_usuario" o "id_tecnologia". Esto hacía que los JOIN que intentaban unir tablas usando esos nombres de columna fallaran al no encontrar las columnas referenciadas.

La solución fue utilizar el comando DESCRIBE nombre_tabla en DBeaver para cada tabla implicada en las consultas con error. Este comando muestra la estructura real de la tabla: los nombres exactos de todas sus columnas, su tipo de datos, si admiten valores nulos y cuáles son las claves. Con esta información se corrigieron todas las consultas, sustituyendo los nombres de columna incorrectos por los reales. Esta experiencia refuerza la importancia de verificar siempre la estructura real de la base de datos antes de escribir consultas, en lugar de asumir que los nombres son los esperados.

8.4 Repositorio no detectado por GitHub Desktop

Al intentar gestionar los commits mediante GitHub Desktop, la aplicación mostraba el error "Can't find LoveCode_Final", indicando que no podía localizar la carpeta del repositorio local en la ubicación donde la tenía registrada. Esto podía deberse a que la carpeta había sido movida o renombrada, o a algún problema de configuración interna de GitHub Desktop.

Al no poder resolver el problema con GitHub Desktop, se optó por una alternativa más directa: subir los archivos al repositorio remoto de GitHub directamente a través de la interfaz web. GitHub permite navegar por las carpetas del repositorio y cargar archivos desde el navegador sin necesidad de ninguna aplicación adicional. Aunque este método no genera el mismo historial de commits detallado que el flujo normal de Git, permitió tener todo el código del proyecto disponible en el repositorio remoto para que pueda ser revisado.

9. Conclusiones

El proyecto LoveCode ha supuesto la experiencia más completa e integradora del ciclo formativo hasta el momento. Por primera vez se ha tenido que diseñar, desarrollar y poner en funcionamiento una aplicación web real de principio a fin, coordinando el trabajo de cinco asignaturas diferentes y enfrentándose a problemas técnicos reales que requirieron soluciones creativas.

Desde el punto de vista técnico, los logros del proyecto son significativos. Se ha diseñado e implementado una base de datos relacional completa con cinco tablas correctamente relacionadas, usuarios con permisos diferenciados, procedimientos almacenados, un trigger automático que genera matches sin intervención del servidor y ocho consultas SQL de diferente complejidad. Se ha desarrollado un servidor backend funcional en Java con Spring Boot con una API REST que cubre todas las funcionalidades de la aplicación. Se ha creado una interfaz web atractiva y completamente funcional que se comunica con el servidor de forma asíncrona. Se ha configurado un sistema de backup automatizado con tarea programada. Y todo esto se ha gestionado con Git y GitHub como herramientas de control de versiones.

Desde el punto de vista del aprendizaje, el proyecto ha demostrado que los conocimientos adquiridos en cada asignatura no son compartimentos estancos sino que se complementan y se necesitan mutuamente. No se puede hacer un buen backend sin entender la base de datos. No se puede hacer una buena interfaz sin entender cómo funciona el backend. Y no se puede mantener un sistema en producción sin tener en cuenta la gestión del entorno y los backups.

Los problemas surgidos durante el desarrollo, lejos de ser un obstáculo negativo, han resultado ser una de las partes más formativas del proyecto. Resolver la pérdida de la máquina virtual migrando a Docker, depurar los errores de dependencias de Maven o corregir los fallos en las consultas SQL son situaciones que cualquier desarrollador profesional puede encontrarse en su trabajo diario, y haberlas experimentado y resuelto en un entorno académico es un aprendizaje muy valioso.

Como posibles líneas de mejora y evolución futura del proyecto se identifican las siguientes:

- Implementar autenticación segura mediante tokens JWT (JSON Web Tokens) para proteger los endpoints de la API y gestionar las sesiones de usuario de forma más robusta y estándar.
- Añadir un sistema de mensajería en tiempo real entre usuarios que han hecho match, usando tecnologías como WebSockets para la comunicación bidireccional.
- Crear una página de perfil personal donde cada usuario pueda editar su información, actualizar su descripción y modificar sus tecnologías.
- Implementar un sistema de notificaciones que avise al usuario cuando tiene nuevos matches o mensajes pendientes.

- Desplegar la aplicación en un servidor en la nube, como AWS, Azure o Google Cloud, para que sea accesible desde cualquier dispositivo con conexión a internet.
- Utilizar Docker Compose para definir todos los servicios del proyecto (base de datos y backend) en un solo archivo de configuración, de forma que todo el entorno se pueda levantar con un único comando.

En definitiva, LoveCode ha cumplido y superado los objetivos planteados al inicio del proyecto. Es una aplicación web funcional, con una base de datos bien diseñada, un backend operativo, una interfaz usable y un sistema de mantenimiento automatizado. Más allá del resultado final, el proceso de desarrollarlo ha consolidado los conocimientos del ciclo formativo y ha demostrado la capacidad de trabajar con tecnologías reales de la industria del desarrollo de software.

10. Referencias

A continuación se listan las principales fuentes, herramientas y tecnologías utilizadas durante el desarrollo del proyecto LoveCode:

Docker Inc. (2024). Docker Documentation. <https://docs.docker.com/>

DBeaver Corp. (2024). DBeaver Community Edition — Universal Database Tool.
<https://dbeaver.io/>

GitHub Inc. (2024). GitHub Documentation. <https://docs.github.com/>

MariaDB Foundation. (2024). MariaDB Server Documentation.
<https://mariadb.com/kb/en/documentation/>

MDN Web Docs. (2024). JavaScript Reference. Mozilla Developer Network.
<https://developer.mozilla.org/es/docs/Web/JavaScript>

Oracle Corporation. (2024). Java SE Development Kit 21. <https://www.oracle.com/java/>

VMware, Inc. (2024). Spring Boot Reference Documentation (Version 3.x).
<https://docs.spring.io/spring-boot/docs/current/reference/html/>

XTART Formación Profesional. (2026). Proyecto LoveCode Web Edition — Documentación del proyecto 3.^a evaluación. Material de clase no publicado.